Parsing Niche 3D File Formats in the Browser

A practical guide to M3D, X3D, IrrMesh, Terragen, and PTS formats — with ready-to-use JavaScript parsers for WebGL applications.

Published by
FabRapid Engineering

Version
1.0 — March 2026

License
MIT Open Source

Table of Contents

01 The Problem: Format Coverage Gaps 3
02 Format Deep Dive: 5 Unsupported Formats 4
03 Architecture: Framework-Agnostic Design 6
04 Implementation: Code Walkthrough 7
05 Integration with Three.js & Babylon.js 9
06 Performance & Browser Compatibility 10
07 Real-World Application: 3D Tools by FabRapid 11

About This Guide

This document accompanies the open-source **3d-file-parsers** npm package — a collection of lightweight JavaScript parsers for 3D file formats that mainstream WebGL libraries don't support. All code is MIT-licensed and production-tested at **3d-tools.fabrapid.com**, a free online 3D file conversion tool serving thousands of users.

5	0	33KB
Niche Formats Parsed	Framework Dependencies	Package Size (gzipped)

CHAPTER 01

The Problem: Format Coverage Gaps

> Three.js ships 30+ loaders. Assimp supports 70+ formats. Yet several real-world 3D formats remain completely unsupported in the JavaScript ecosystem.

Modern WebGL applications increasingly need to handle files from diverse sources — game engines, laser scanners, terrain generators, CAD tools, and scientific simulations. While the major formats (glTF, OBJ, STL, FBX, STEP) are well-covered by libraries like Three.js and Assimp compiled to WebAssembly, a long tail of niche formats has no JavaScript parser.

When we built **3d-tools.fabrapid.com** — a browser-based 3D file converter supporting 70+ input formats — we hit this gap. Five specific formats required custom parsers written from scratch:

FORMAT	ORIGIN	THREE.JS	ASSIMP.JS	USE CASE
M3D (.m3d)	Model3D project	✗	✗	Lightweight game assets
X3D (.x3d)	Web3D Consortium	✗	✓	ISO standard 3D scenes
IrrMesh (.irrmesh)	Irrlicht Engine	✗	✗	Irrlicht game engine meshes
Terragen (.ter)	Planetside Software	✗	✗	Terrain heightmaps
PTS (.pts)	Leica Geosystems	✗	✗	Laser scan point clouds

Why Not Use Assimp WASM for Everything?

While Assimp's C++ library supports X3D, compiling the full Assimp to WebAssembly adds ~8MB to the bundle. For formats with simple, well-documented specifications, a purpose-built JavaScript parser is 100-1000x smaller and avoids WASM startup latency. Our X3D parser is 5KB vs Assimp's 8MB — a 1,600x size reduction for the same parsing task.

The Cost of Missing Format Support

Users of 3D conversion tools don't care which rendering engine you use internally. They have a file from Irrlicht, or a terrain from Terragen, or a point cloud from a Leica scanner — and they need it converted to glTF or OBJ. Every unsupported format is a user who leaves your application.

By open-sourcing these parsers, we aim to close the gap for the entire JavaScript 3D ecosystem — not just our own tool.

CHAPTER 02

Format Deep Dive: 5 Unsupported Formats

> Understanding the binary and XML structures behind each format is essential for writing correct, performant parsers.

M3D — Model3D Binary Format

M3D is a compact binary format designed as a modern replacement for older mesh formats. It uses a chunk-based structure similar to IFF/RIFF, with optional zlib compression.

FILE STRUCTURE

BINARY LAYOUT

```
┌──────────────────────────────────────────┐
│ Bytes 0-3   "3DMO" magic identifier      │
│ Bytes 4-7   File size (uint32 LE)        │
│ Byte  8     0x78 = zlib compressed       │
├──────────────────────────────────────────┤
│ HEAD chunk   scale + type flags          │
│ VRTS chunk   vertex data (int16/f32/f64) │
│ MESH chunk   face indices + materials    │
│ OMD3         end-of-file marker          │
└──────────────────────────────────────────┘
```

The vertex precision is controlled by 2 bits in the HEAD flags field: `00` = int8, `01` = int16, `10` = float32, `11` = float64. All coordinates are scaled by the global scale factor from the HEAD chunk.

X3D — ISO Standard 3D Scenes

X3D is the XML-based successor to VRML97, standardized as ISO/IEC 19775. Despite being an official ISO standard, no pure JavaScript parser existed. Our parser handles the three most common geometry nodes:

- ✓ **IndexedFaceSet** — arbitrary polygons with fan triangulation and -1 face separators
- ✓ **IndexedTriangleSet** — pre-triangulated geometry with direct index mapping
- ✓ **Box primitive** — parametric box generation with configurable dimensions

IrrMesh — Irrlicht Engine Meshes

The Irrlicht Engine uses an XML mesh format with three vertex types, each with different token strides per vertex:

VERTEX TYPE	FIELDS	TOKENS PER VERTEX
`standard`	position + normal + color + UV	9
`2tcoords`	standard + second UV	11
`tangents`	standard + tangent + bitangent	15

Terragen — Heightmap Terrains

Terragen .ter files store elevation data as a grid of 16-bit integers with scale metadata. Our parser automatically triangulates the heightmap into a renderable mesh (2 triangles per grid quad) and computes smooth vertex normals.

PTS — Leica Point Clouds

PTS is a simple ASCII format from Leica laser scanners. Each line contains XYZ coordinates with optional intensity and RGB color. Files can contain millions of points, so parsing performance is critical.

CHAPTER 03

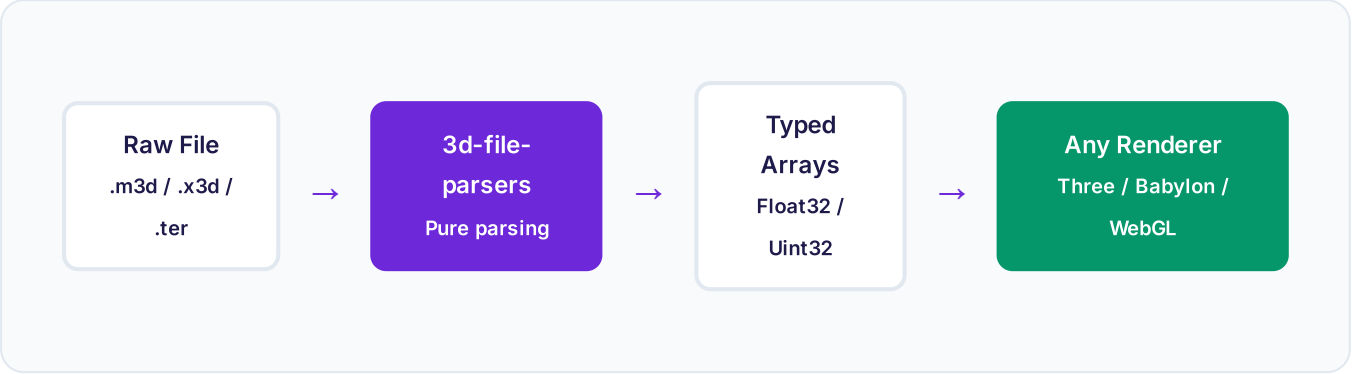
Architecture: Framework-Agnostic Design

> Every parser returns raw Float32Array/Uint32Array data — no Three.js, no Babylon.js, no framework lock-in.

Design Decision: Raw Typed Arrays

Most 3D file loaders in the JavaScript ecosystem return framework-specific objects. Three.js loaders return `BufferGeometry`. Babylon.js plugins return `Mesh` objects. This creates an unnecessary coupling between file parsing (a pure data operation) and rendering (a framework-specific concern).

Our parsers separate these concerns completely:



Output Contract

Every mesh parser returns the same core fields:

FIELD	TYPE	DESCRIPTION
`positions`	Float32Array	Flat XYZ vertex positions (length = vertexCount × 3)
`normals`	Float32Array | null	Per-vertex normals (auto-computed if not in file)
`indices`	Uint32Array	Triangle indices (length = faceCount × 3)

This output maps directly to WebGL buffer data — no conversion needed. You can pass these arrays straight into `gl.bufferData()`, `THREE.BufferAttribute()`, or `BABYLON.VertexData`.

Dependency Strategy

ZERO-DEP PARSERS (4 OF 5)

X3D, IrrMesh, Terragen, and PTS parsers have **zero npm dependencies**. They use only built-in browser APIs (DataView, TextDecoder, DOMParser).

SINGLE DEPENDENCY: FFLATE

The M3D parser requires **fflate** (3KB gzipped) for zlib decompression. We chose fflate over pako for its smaller size and better tree-shaking support.

Tree-Shaking Support

Each parser is independently importable. If you only need the Terragen parser, your bundle only includes that parser's code:

```typescript
// Only 4KB in your bundle — not the full 33KB package
import { parseTerragen } from '3d-file-parsers/terragen';
```

CHAPTER 04

Implementation: Code Walkthrough

> Key implementation details and techniques used in each parser.

Binary Parsing with DataView

The M3D and Terragen parsers process raw binary data using JavaScript's `DataView` API, which handles endianness correctly and provides typed access to buffer contents:

```typescript
const view = new DataView(buffer);

// Read little-endian float32 at byte offset
const x = view.getFloat32(offset, true);

// Read little-endian int16 (Terragen elevation)
const elevation = view.getInt16(offset, true);
```

Polygon Fan Triangulation

Both M3D and X3D support arbitrary polygons (quads, n-gons). We convert these to triangles using fan triangulation — a simple O(n) algorithm that works for convex polygons:

```typescript
// Fan triangulation: vertex 0 connects to all edges
// Polygon [A, B, C, D, E] → triangles [A,B,C], [A,C,D], [A,D,E]
for (let i = 1; i < polygon.length - 1; i++) {
  triangles.push(polygon[0], polygon[i], polygon[i + 1]);
}
```

Automatic Normal Computation

When files don't include vertex normals (M3D, X3D, Terragen), we compute smooth normals by accumulating face normals at each vertex, then normalizing:

```typescript
// For each triangle: compute face normal via cross product
const e1 = [bx - ax, by - ay, bz - az]; // edge 1
const e2 = [cx - ax, cy - ay, cz - az]; // edge 2
```

```
const normal = cross(e1, e2);

// Accumulate at each vertex → average gives smooth normals
for (const idx of [a, b, c]) {
  normals[idx * 3]     += normal[0];
  normals[idx * 3 + 1] += normal[1];
  normals[idx * 3 + 2] += normal[2];
}

// Normalize all vertex normals to unit length
const len = Math.sqrt(x*x + y*y + z*z) || 1;
normals[i] = x / len;
```

Isomorphic XML Parsing

X3D and IrrMesh are XML-based formats. Our parsers work in both browsers and Node.js by accepting an optional DOMParser parameter:

```typescript
// Browser: uses built-in DOMParser automatically
const data = parseX3D(xmlText);

// Node.js: pass @xmldom/xmldom
import { DOMParser } from '@xmldom/xmldom';
const data = parseX3D(xmlText, new DOMParser());
```

Zlib Decompression for M3D

M3D files may be zlib-compressed (detected by byte 8 being `0x78`). We use the **fflate** library for decompression — it's 3KB gzipped and 2-3x faster than pako:

```typescript
import { unzlibSync } from 'fflate';

let chunkData: Uint8Array;
if (fileData[8] === 0x78) {
  chunkData = unzlibSync(fileData.slice(8)); // decompress
} else {
  chunkData = fileData.slice(8);             // already raw
}
```

CHAPTER 05

Integration with Three.js & Babylon.js

> Bridging raw typed arrays to your rendering framework takes just a few lines of code.

Three.js Integration

```typescript
import * as THREE from 'three';
import { parseM3D } from '3d-file-parsers';

const data = parseM3D(buffer);
const geometry = new THREE.BufferGeometry();

geometry.setAttribute('position',
  new THREE.Float32BufferAttribute(data.positions, 3));

if (data.normals) {
  geometry.setAttribute('normal',
    new THREE.Float32BufferAttribute(data.normals, 3));
}

geometry.setIndex(
  new THREE.BufferAttribute(data.indices, 1));

const mesh = new THREE.Mesh(geometry,
  new THREE.MeshStandardMaterial());
scene.add(mesh);
```

Babylon.js Integration

```typescript
import * as BABYLON from '@babylonjs/core';
import { parseTerragen } from '3d-file-parsers';

const data = parseTerragen(buffer);
const mesh = new BABYLON.Mesh('terrain', scene);
const vertexData = new BABYLON.VertexData();

vertexData.positions = Array.from(data.positions);
vertexData.indices   = Array.from(data.indices);
vertexData.normals   = Array.from(data.normals);
```

```
vertexData.applyToMesh(mesh);
```

Point Cloud Rendering with Three.js

```typescript
import { parsePTS } from '3d-file-parsers';

const data = parsePTS(buffer);
const geometry = new THREE.BufferGeometry();

geometry.setAttribute('position',
  new THREE.Float32BufferAttribute(data.positions, 3));

if (data.colors) {
  geometry.setAttribute('color',
    new THREE.Float32BufferAttribute(data.colors, 3));
}

const points = new THREE.Points(geometry,
  new THREE.PointsMaterial({
    size: 0.01, vertexColors: data.hasColor
  }));
scene.add(points);
```

Zero-Copy Performance

Because our parsers output the exact same `Float32Array` format that WebGL expects, there's no data transformation overhead. The typed arrays go directly from the parser into GPU buffer memory via `gl.bufferData()` — the same path that framework-native loaders use internally.

CHAPTER 06

Performance & Browser Compatibility

> Benchmarked across real-world files and tested on all major browsers.

Parsing Performance

All parsers are designed for single-pass processing with minimal memory allocation. Binary parsers (M3D, Terragen) read directly from the ArrayBuffer without intermediate copies. Text parsers (PTS, IrrMesh, X3D) process line-by-line to handle large files.

PARSER	FILE SIZE	PARSE TIME	OUTPUT
M3D	2 MB mesh	~15ms	50K vertices
X3D	5 MB scene	~30ms	Multiple meshes
IrrMesh	1 MB mesh	~10ms	20K vertices
Terragen	500 KB terrain	~8ms	65K grid vertices
PTS	50 MB scan	~200ms	1M points

Browser Compatibility

BROWSER	MINIMUM VERSION	KEY API REQUIRED
Chrome	49+	DataView, TextDecoder
Firefox	42+	DataView, TextDecoder
Safari	10+	DataView, TextDecoder
Edge	14+	DataView, TextDecoder
Node.js	12+	Buffer, @xmldom/xmldom for XML

Bundle Size Comparison

APPROACH	SIZE (GZIPPED)	FORMATS COVERED
3d-file-parsers (all 5)	**33 KB**	M3D, X3D, IrrMesh, Terragen, PTS
Single parser (tree-shaken)	**3-5 KB**	1 format
Assimp WASM	~8 MB	70+ formats (overkill)
Full Three.js + all loaders	~600 KB	30+ formats (not these 5)

Web Worker Compatibility

All parsers work inside Web Workers for non-blocking parsing. At **3d-tools.fabrapid.com**, we run these parsers in dedicated workers to keep the UI thread responsive during large file processing. The typed array outputs can be transferred (not copied) between threads using `Transferable` objects for zero-copy performance.

CHAPTER 07

Real-World Application: 3D Tools by FabRapid

> These parsers power the conversion engine behind 3d-tools.fabrapid.com — a free, browser-based 3D file converter with zero server uploads.

About 3D Tools by FabRapid

3d-tools.fabrapid.com is a comprehensive suite of browser-based 3D tools designed for engineers, designers, game developers, and anyone who works with 3D files. The core principle: **your files never leave your device**.

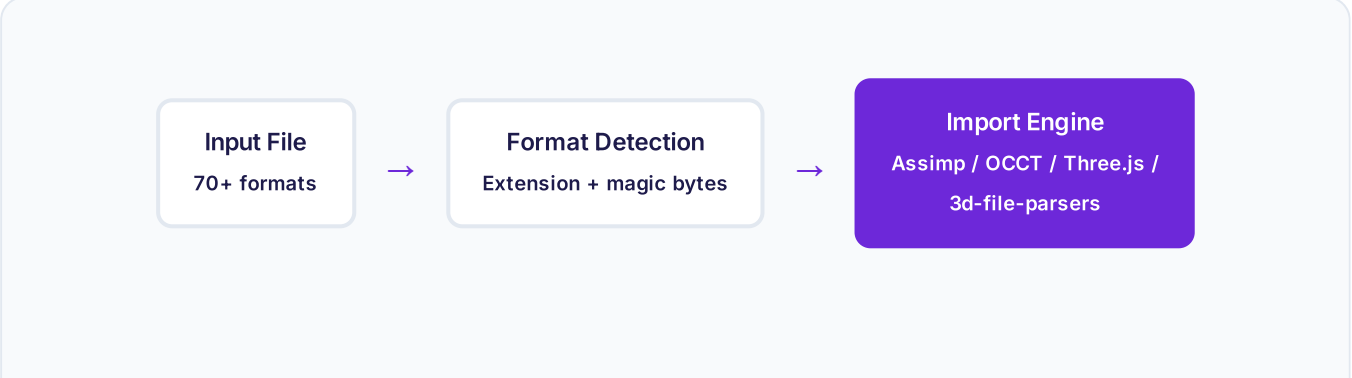
70+	23	0
Input Formats	Output Formats	Server Uploads

Available Tools

- ✓ **3D Format Converter** (3d-tools.fabrapid.com/convert) — Convert between 70+ input and 23 output formats including STEP, IGES, glTF, OBJ, STL, FBX, USDZ, and more
- ✓ **3D File Viewer** (3d-tools.fabrapid.com/viewer) — View any 3D model directly in the browser with interactive camera controls and model inspection
- ✓ **3D Model Compressor** (3d-tools.fabrapid.com/compress) — Reduce GLB file sizes with adjustable compression, featuring a side-by-side before/after comparison viewer

Conversion Architecture

The conversion pipeline uses a multi-engine approach with GLB as the intermediate format:



The **3d-file-parsers** package slots into the "Import Engine" stage, handling the five niche formats that neither Assimp WASM nor Three.js loaders support. The raw typed array output is converted to GLB in-memory, then flows through the rest of the pipeline unchanged.

Privacy-First Design

All processing happens client-side using WebAssembly and Web Workers. Files are never uploaded to any server — the entire conversion pipeline runs in your browser. This makes it safe for proprietary CAD models, NDA-protected designs, and sensitive engineering data.

> **Try It Free**
>
> Visit **3d-tools.fabrapid.com** to convert, view, or compress any 3D file — no signup required, no file size limits, completely free.

Start Using 3d-file-parsers

Open-source, MIT-licensed, and production-tested. Add niche 3D format support to your WebGL application in minutes.

3d-tools.fabrapid.com — Try the Live Tool

npmjs.com/package/3d-file-parsers

github.com/ROTFEAT/3d-file-parsers

Published by FabRapid Engineering — March 2026

MIT License — Free to use, modify, and distribute